

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 4 + 16

Date : 17/03/2023
Matière : Programmation Java 2
Durée : 1h
Documents : Non autorisés
Nombre des pages : 5

Devoir Surveillé

Exercice 1

Qu'affiche le programme suivant ?

```
import java.util.*;
public class TestDevoir {
    public static void main(String[] args) {
        List<String> l0 = Arrays.asList("maman", "bateau", "ananas");
        List<String> l1 = List.copyOf(l0);
        try {
            l0.set(0, "aabam");
            System.out.println("Affichage 1");
            l0.forEach(System.out::println);
            System.out.println("Affichage 2");
            l1.forEach(System.out::println);
            l1.add("avion");
            System.out.println("Affichage 3");
            l1.forEach(System.out::println);
        } catch (UnsupportedOperationException e) {
            System.err.println("Opération non supportée");
        }
        l1.stream()
            .map(e -> e.substring(1, 3))
            .filter(c -> c.startsWith("a"))
            .forEach(System.out::println);
    }
}
```

Affichage 1

aabam

bateau

ananas

Affichage 2

maman

bateau

ananas

Opération non supportée

am

at

Exercice 2

On souhaite modéliser un registre de patients qui sont les clients d'une pharmacie.

Chaque patient est représenté par son nom, son code et son âge. Deux patients sont égaux s'ils ont le même nom, le même code et le même âge.

Un médicament est représenté par son nom, sa quantité et son prix unitaire. On suppose que les médicaments sont comparés uniquement selon leurs prix unitaires.

Un registre pharmaceutique est caractérisé par un registre de type Map (la partie clé est Patient, la partie valeur est un ArrayList de Medicament) qui permet de créer initialement une Map vide tout en conservant l'ordre d'insertion des éléments qui seront ajoutés par la suite.

Une partie du code des classes **Patient**, **Medicament** et **RegistrePharmacie** vous est fournie. A vous de compléter le corps des méthodes signalées.

1. Compléter la classe **Patient** donnée ci-dessous.

```
import java.util.*;
class Patient {
    private String nom;
    private int code;
    private int age;

    public Patient(String nom, int code, int age) {
        this.nom = nom;
        this.code = code;
        this.age = age;
    }
    public String getNom() {
        return nom;
    }
    public void setNom(String nom) {
        this.nom = nom;
    }
    public int getCode() {
        return code;
    }
    public void setCode(int code) {
        this.code = code;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }

    @Override
    public int hashCode() {
        // A compléter
        return Objects.hash(age, code, nom);
    }

    public boolean equals(Object obj) {
        // A compléter
    }
}
```

```
if (this == obj)
    return true;
if (obj == null || obj.getClass() != this.getClass())
    return false;
Patient other = (Patient) obj;
return age == other.age && code == other.code &&
Objects.equals(nom, other.nom);
}

    public String toString() {
        return "Patient : "+getNom()+" : "+getCode()+" : "+getAge();
    }
}
```

2. Compléter la classe **Medicament** donnée ci-dessous.

```
import java.util.*;
class Medicament implements Comparable<Medicament> {
    private String nomMed;
    private int quantite;
    private double prix;

    public Medicament(String nomMed, int quantite, double prix) {
        this.nomMed = nomMed;
        this.quantite = quantite;

        this.prix = prix;
    }
    public String getNomMed() {
        return nomMed;
    }
    public void setNomMed(String nomMed) {
        this.nomMed = nomMed;
    }
    public int getQuantite() {
        return quantite;
    }
    public void setQuantite(int quantite) {
        this.quantite = quantite;
    }
    public double getPrix() {
        return prix;
    }
    public void setPrix(double prix) {
        this.prix = prix;
    }
}

/* Retourne un ArrayList d'un médicament créé selon le constructeur de cette
classe */
    public ArrayList<Medicament> getListMedicament() {
        // A compléter
        ArrayList<Medicament> l = new ArrayList<>();
        //l.add(new Medicament(this.getNomMed(), this.getQuantite(),
        //this.getPrix()));
        return l;
    }
}
```

```
/* Retourne un tableau de chaînes de caractères (tab[0]=nomMed, tab[1]=quantite et
tab[2]=prix) */
public String[] getTabMedicament() {
    // A compléter
    String[] tab = new String[3];
    tab[0] = this.getNomMed();
    tab[1] = String.valueOf(this.getQuantite());
    tab[2] = String.valueOf(this.getPrix());

    // String[] tab = new String[]{this.getNomMed(),
    // String.valueOf(this.getQuantite()),
    // String.valueOf(this.getPrix())};

    return tab;
}

@Override
public int hashCode() {
    // A compléter
    return Objects.hash(prix);
}

@Override
public boolean equals(Object obj) {
    // A compléter
    if (this == obj)
        return true;
    if (obj == null || obj.getClass() != this.getClass())
        return false;
    Medicament other = (Medicament) obj;
    return prix == other.prix;
}

@Override
public int compareTo(Medicament o) {
    // A compléter
    return Double.valueOf(prix).compareTo(o.prix);
}

public String toString() {
    return "Medicament : "+getNomMed()+" : "+getQuantite()+" : "+getPrix();
}
}
```

3. Compléter la classe **RegistrePharmacie** signalée ci-dessous.

```
import java.util.*;
class RegistrePharmacie {

    Map<Patient, ArrayList<Medicament>> registre;

    public RegistrePharmacie() {

        // A compléter
        registre = new LinkedHashMap<>();
    }
}
```

```

/* Ajoute dans le registre un patient p ayant une ordonnance d'un médicament ord
sous forme d'un tableau de chaînes de caractères (ord[0]=nomMed,
ord[1]=quantite et ord[2]=prix) */
public void ajoutPatientMedicament(Patient p, String[] ord) {
    // A compléter
    Medicament med = new
    Medicament(ord[0],Integer.parseInt(ord[1]),Double.parseDouble(ord[2]));
    if(! registre.containsKey(p)) {
        ArrayList<Medicament> m1 = new ArrayList<>();
        m1.add(med);
        registre.put(p, m1);
    }
    else {
        ArrayList<Medicament> li = registre.get(p);
        li.add(med);
    }
}

/* Affiche les noms des patients du registre */
public void afficheNomPatient() {
    // A compléter
    Set<Patient> setp =registre.keySet();
    for (Patient pp:setp)
        System.out.println(pp.getNom());

    // A compléter
    // registre.forEach((k,v)-> System.out.println(k.getNom()));
}

/* Retourne une liste des noms des médicaments pris par le (les) patient(s) ayant
le nom nomPatient donné */
public List<String> getNomMedPatient(String nomPatient) {
    // A compléter
    List<String> mm = new ArrayList<>();
    Set< Map.Entry< Patient, ArrayList<Medicament> > > entry =
    registre.entrySet();
    for (Map.Entry<Patient, ArrayList<Medicament>> pp : entry) {
        if( pp.getKey().getNom().equals(nomPatient)) {
            for (Medicament medd : pp.getValue())
                mm.add(medd.getNomMed());
        }
    }
    return mm;

    // A compléter
    // List<String> mm = new ArrayList<>();
    // for (var pp : registre.entrySet()) {
    //     if( pp.getKey().getNom().equals(nomPatient)) {
    //         for (Medicament medd : pp.getValue())
    //             mm.add(medd.getNomMed());
    //     }
    // }
    // return mm;
}

```

```
// List<String> mm = new ArrayList<>();
// for (Patient p: registre.keySet()){
//     if(p.getNom().equals(nomPatient))
//         for (Medicament medd : registre.get(p))
//             mm.add(medd.getNomMed());
// }
// return mm;
}

/* Retourne une liste triée contenant les codes des patients prenant le médicament
nomMedicament */
public List<Integer> getCodePatientMed(String nomMedicament){
    // A compléter
    List<Integer> se =new ArrayList<>();
    for (Patient pp: registre.keySet()){
        for(Medicament mm : registre.get(pp)){
            if(mm.getNomMed().equals(nomMedicament)) {
                se.add(pp.getCode());
                break;
            }
        }
    }
    Collections.sort(se);
    return se;

    // A compléter
    // List<Integer> se =new ArrayList<>();
    // for (var pp : registre.entrySet()) {
    //     for(Medicament mm : pp.getValue()){
    //         if(mm.getNomMed().equals(nomMedicament)) {
    //             se.add(pp.getKey().getCode());
    //             break;
    //         }
    //     }
    // }
    // Collections.sort(se);
    // return se;
}

/* Retourne le prix total des médicaments d'un patient p */
public double getPrixMedicament(Patient p){
    // A compléter
    double solde=0;

    for(var mp : registre.entrySet()) {
        if(mp.getKey().equals(p)) {
            for(Medicament ms: mp.getValue())
                solde = solde + (ms.getQuantite()*ms.getPrix());
        }
    }
    return solde;
}
```

```
// A compléter
// double solde=0;
// for(Medicament ms: registre.get(p))
//     solde = solde + (ms.getQuantite()*ms.getPrix());
// return solde;
}

/* Retourne une liste des noms des patients ayant le prix total des médicaments le
   plus élevé */
public List<String> getNomPatientMed(){
    // A compléter
    double pr=0;
    List<String> s = new ArrayList<>();
    for (Patient p1: registre.keySet()){
        if (pr < this.getPrixMedicament(p1))
            pr=this.getPrixMedicament(p1);
    }

    for (Patient p2: registre.keySet()){
        if (pr == this.getPrixMedicament(p2))
            s.add(p2.getNom());
    }
    return s;
}

/* Retourne le nombre de patients qui ont un âge supérieur ou égal au paramètre
   seuil donné. Pour un seuil non entier, cette méthode doit retourner -1 */
public int getCountPatientAge(String seuil){
    // A compléter
    int res =0;
    int val;
    try {
        val =Integer.parseInt(seuil);
    }catch(NumberFormatException e) {
        return -1;
    }

    if(val <0)
        return -1;

    for (Patient p: registre.keySet()){
        if(p.getAge() >= val)
            res++;
    }
    return res;
}
}
```

BON TRAVAIL